

Exp: 7 Write a C program to find FIRST() - predictive parser for the given grammar

```
#include <stdio.h>

#include <ctype.h>

void FIRST(char[], char);
void addToResultSet(char[], char);

int numOfProductions;
char productionSet[10][10];

int main() {
    int i;
    char choice;
    char c;
    char result[20];

    printf("How many number of productions? : ");
    scanf(" %d", &numOfProductions);

    for (i = 0; i < numOfProductions; i++) {
        printf("Enter production Number %d (e.g., E=E+T): ", i + 1);
        scanf(" %s", productionSet[i]);
    }

    do {
        printf("\nFind the FIRST of: ");
        scanf(" %c", &c);

        FIRST(result, c); // Compute FIRST; Get Answer in 'result' array
```

```

    printf("\nFIRST(%c) = { ", c);
    for (i = 0; result[i] != '\0'; i++) {
        printf(" %c ", result[i]);
    }
    printf("}\n");

    printf("Press 'y' to continue: ");
    scanf(" %c", &choice);

} while (choice == 'y' || choice == 'Y');

return 0;
}

void FIRST(char* Result, char c) {
    int i, j, k;
    char subResult[20];
    int foundEpsilon;

    subResult[0] = '\0';
    Result[0] = '\0';
    if (!isupper(c)) {
        addToResultSet(Result, c);
        return;
    }
    for (i = 0; i < numOfProductions; i++) {
        if (productionSet[i][0] == c) {
            if (productionSet[i][2] == '$') {
                addToResultSet(Result, '$');
            } else {
                j = 2;
                while (productionSet[i][j] != '\0') {

```

```

        foundEpsilon = 0;

        FIRST(subResult, productionSet[i][j]);

        for (k = 0; subResult[k] != '\0'; k++) {
            addToResultSet(Result, subResult[k]);
        }
        for (k = 0; subResult[k] != '\0'; k++) {
            if (subResult[k] == '$') {
                foundEpsilon = 1;
                break;
            }
        }
        if (!foundEpsilon) {
            break;
        }
        j++;
    }
}

return;
}

void addToResultSet(char Result[], char val) {
    int k;

    for (k = 0; Result[k] != '\0'; k++) {
        if (Result[k] == val) {
            return;
        }
    }

    Result[k] = val;
    Result[k + 1] = '\0';
}

```

}

Output:

```
How many number of productions? : 4
Enter production Number 1 (e.g., E=E+T): S=AaAb
Enter production Number 2 (e.g., E=E+T): S=BbBa
Enter production Number 3 (e.g., E=E+T): $
Enter production Number 4 (e.g., E=E+T): $

Find the FIRST of: S

FIRST(S) = { }
Press 'y' to continue: Y

Find the FIRST of: A

FIRST(A) = { }
Press 'y' to continue: Y

Find the FIRST of: B

FIRST(B) = { }
Press 'y' to continue: N

...Program finished with exit code 0
Press ENTER to exit console.
```